

Лекция 12.

Итоговый инфраструктурный проект Linux-инфраструктуры

Цель лекции

Связать технологии модуля в одну управляемую инфраструктуру

Разделить обязательную и расширенную часть проекта

Согласовать topology, address plan, DNS, firewall и acceptance checks

Подготовить требования к отказным тестам, backup, restore и reboot test

Понять, как защищать проект как работающую систему, а не набор установленных сервисов

Учебные вопросы

1. Архитектура итогового стенда и обязательные роли
2. Address plan, naming, DNS и service names
3. Routing, firewall matrix, VPN и traffic zones
4. Storage, Git, BorgBackup, monitoring и порядок развертывания
5. Failure tests, reboot test и acceptance checks
6. Документация, защита, оценивание и критические ошибки

Главная идея проекта

Infrastructure Project ≠ набор установленных сервисов

Проект проверяет, как сервисы работают вместе

Architecture -> Configuration -> Monitoring -> Failure Test

Recovery -> Reboot Test -> Documentation

Система считается готовой только после проверки
восстановления и работы после перезагрузки

Обязательная и расширенная части не должны смешиваться

Обязательная часть выполняется каждым студентом

Расширенная часть выбирается как один или несколько дополнительных компонентов

Нельзя требовать Mailcow, Docker Application, Keepalived и Separate Backup Network одновременно у всех

Каждый выбранный компонент должен быть отражен в topology, address plan, firewall matrix и acceptance checklist

Требование, которое не проверяется, не должно попадать в критерии оценивания

Обязательная архитектура проекта

client01 - клиентская проверка пользовательского доступа

admin01 - административная рабочая станция

gw01 - Router и Inter-zone Firewall

dns01 - внутренний DNS

vpn01 - OpenVPN

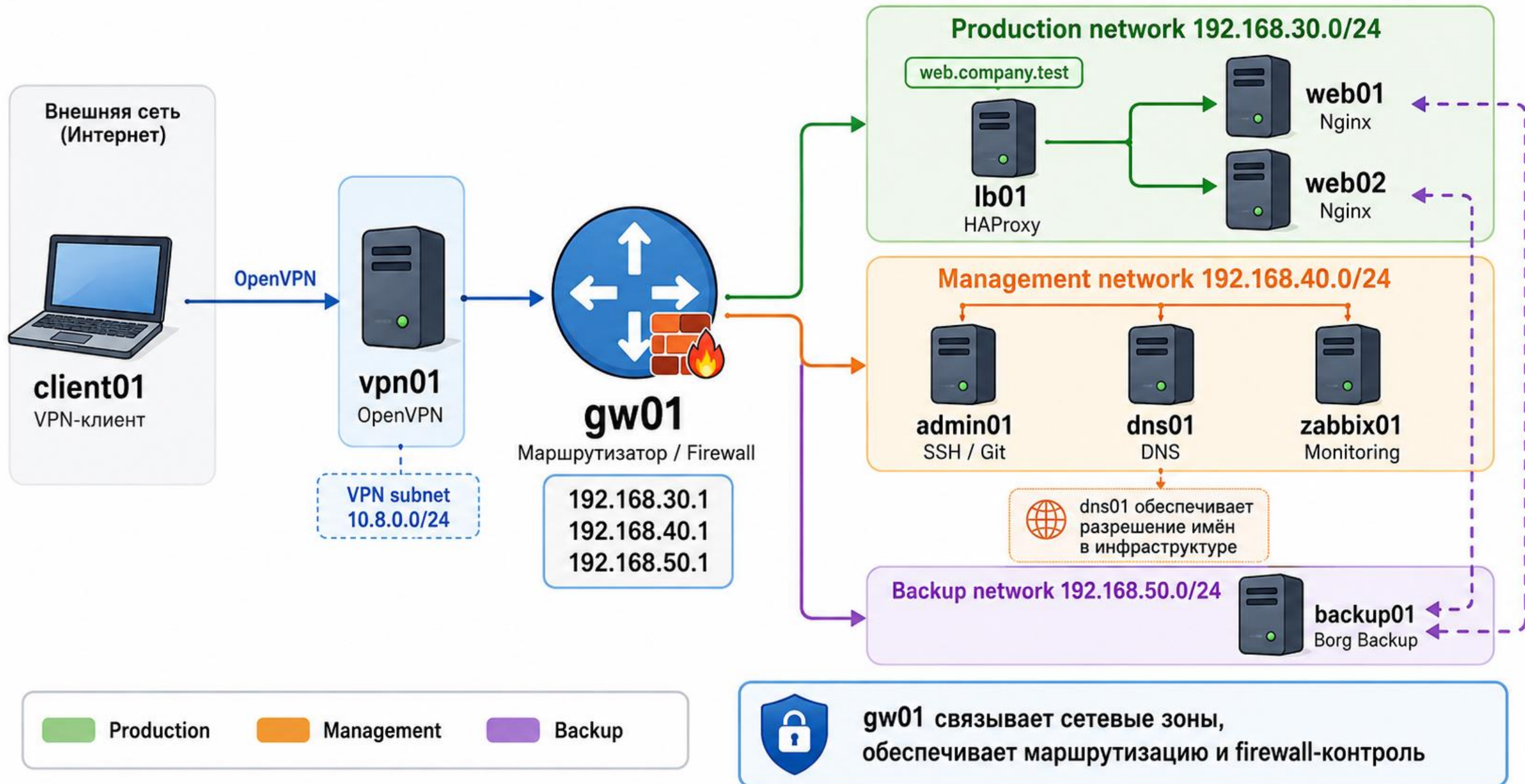
lb01 - HAProxy entry point

web01/web02 - Nginx backend

zabbix01 - monitoring

backup01 - backup repository

Обязательная Linux-инфраструктура с gw01



gw01 явно маршрутизирует traffic между зонами

gw01 выполняет роль Router

gw01 выполняет роль Inter-zone Firewall

Production gateway: 192.168.30.1

Management gateway: 192.168.40.1

Backup gateway: 192.168.50.1, если используется
отдельная Backup Network

Используются изученные технологии: IP forwarding,
nftables и routing

Сетевые зоны должны иметь понятные gateway

Production: 192.168.30.0/24, gateway gw01 192.168.30.1

Management: 192.168.40.0/24, gateway gw01
192.168.40.1

Backup: 192.168.50.0/24, gateway gw01 192.168.50.1,
если выбран расширенный вариант

VPN: 10.8.0.0/24, tunnel gateway vpn01 10.8.0.1

Если отдельная Backup Network не используется,
backup traffic идет по внутренней сети через строгий
firewall

Backup Network может быть базовой или расширенной

Базовый вариант: backup traffic идет через внутреннюю сеть и ограничивается firewall

Расширенный вариант: отдельная Backup Network 192.168.50.0/24

Если Backup Network выбрана, она должна быть на topology

Она должна быть в address plan

Она должна быть в firewall matrix

Нельзя писать о Backup Network в тексте и не показывать ее в проекте

Полный address plan должен быть согласован со схемой

gw01: 192.168.30.1 / 192.168.40.1 / 192.168.50.1 -

Router/Firewall

admin01: 192.168.40.20 - Admin workstation

dns01: 192.168.40.53 - DNS

vpn01: 192.168.30.5 / 192.168.40.5 / 10.8.0.1 - OpenVPN

lb01: 192.168.30.10 / 192.168.40.10 - HAProxy

web01: 192.168.30.11 / 192.168.40.11 / 192.168.50.11 - Nginx

web02: 192.168.30.12 / 192.168.40.12 / 192.168.50.12 - Nginx

zabbix01: 192.168.40.40 - Monitoring

backup01: 192.168.40.50 / 192.168.50.50 - Backup

Host Name и Service Name - разные сущности

Host Name - имя конкретного сервера

Service Name - имя сервиса, которым пользуется клиент

web01.company.test - host name backend-сервера

web02.company.test - host name backend-сервера

web.company.test - service name web-сервиса

web.company.test должен указывать на HAProxy или VIP, а не на случайный backend

Naming standard делает проект читаемым

Использовать формат role + number + company.test

web01.company.test

web02.company.test

lb01.company.test

dns01.company.test

backup01.company.test

Имена должны совпадать в DNS, monitoring,
documentation и командах проверки

Service DNS names используются в пользовательских проверках

web.company.test. IN A 192.168.30.10

web.company.test указывает на HAProxy lb01

Если используется Keepalived/VIP, web.company.test указывает на VIP

Если выбран Mailcow: mail01.company.test - host,
mail.company.test - service

Все external tests должны использовать service names,
а не случайные IP-адреса

DNS role должна быть явно проверена

getent hosts web01.company.test

getent hosts web.company.test

dig +short A lb01.company.test

dig +short A web.company.test

resolvectl query web.company.test - дополнительная команда

Если DNS не работает, следующие проверки web, VPN и monitoring могут вводить в заблуждение

Host Names, Service Names и DNS-записи company.test

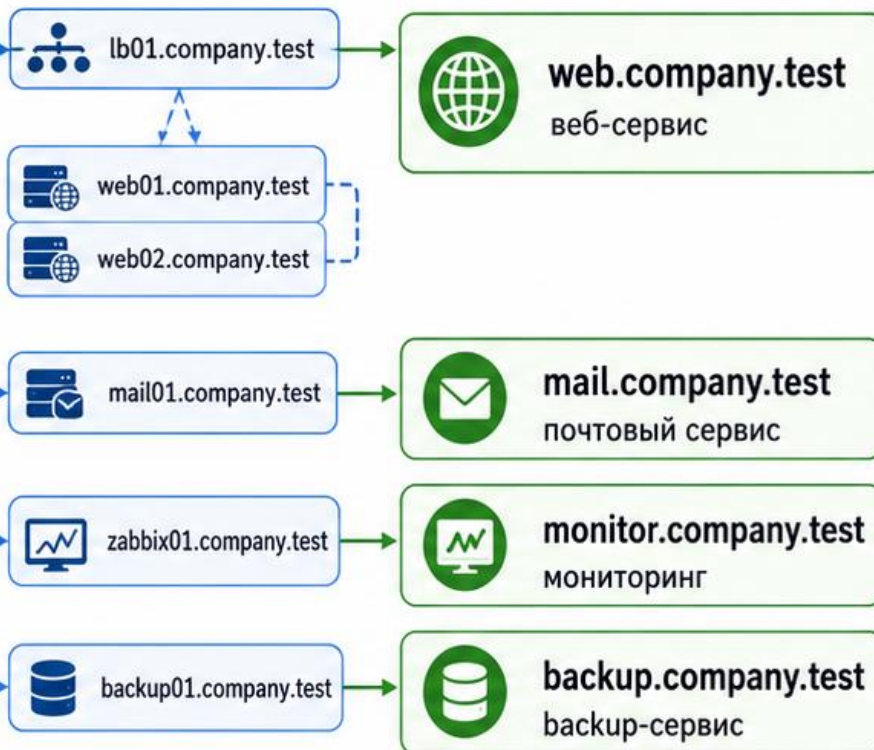
HOST NAMES

Имя конкретного сервера

- gw01.company.test
- vpn01.company.test
- dns01.company.test
- lb01.company.test
- web01.company.test
- web02.company.test
- zabbix01.company.test
- backup01.company.test
- mail01.company.test

SERVICE NAMES

Имя сервиса для пользователя



DNS-ЗАПИСИ

A	web01.company.test	→	192.168.30.11
A	web02.company.test	→	192.168.30.12
A	lb01.company.test	→	192.168.30.10
A	dns01.company.test	→	192.168.40.53
A	zabbix01.company.test	→	192.168.40.40
A	backup01.company.test	→	192.168.50.50
A	web.company.test	→	192.168.30.10
A	mail.company.test	→	192.168.30.20
CNAME	monitor.company.test	→	zabbix01.company.test
CNAME	backup.company.test	→	backup01.company.test
MX	company.test	→	mail.company.test

ГЛАВНАЯ ИДЕЯ



Host Name =
конкретный узел



Service Name =
точка входа сервиса



DNS связывает
имена и IP-адреса



Пользователь обращается
к **service name**,
а не к backend-хостам напрямую.

Traffic zones задают границы безопасности

Production - пользовательские сервисы и backend traffic

Management - SSH, monitoring, admin access

Backup - backup traffic, если выбран отдельный сегмент

VPN - удаленные клиенты

Каждый flow должен иметь Source, Destination, Protocol, Port, Action и Reason

Firewall matrix обязательно включает DNS flows

Production -> dns01 UDP/TCP 53 ALLOW, internal name resolution

Management -> dns01 UDP/TCP 53 ALLOW, admin and monitoring name resolution

VPN -> dns01 UDP/TCP 53 ALLOW, VPN client internal names

Backup -> dns01 UDP/TCP 53 ALLOW, only if separate Backup Network uses DNS

DNS flows добавляются только для сетей, которым реально нужен внутренний DNS

Firewall matrix фиксирует разрешенный и запрещенный доступ

VPN Client -> lb01 TCP 443 ALLOW, web access

VPN Client -> web01/web02 TCP 22 DENY, no direct SSH to backend

admin01 -> servers TCP 22 ALLOW, administration

zabbix01 -> agents TCP 10050 ALLOW, passive checks

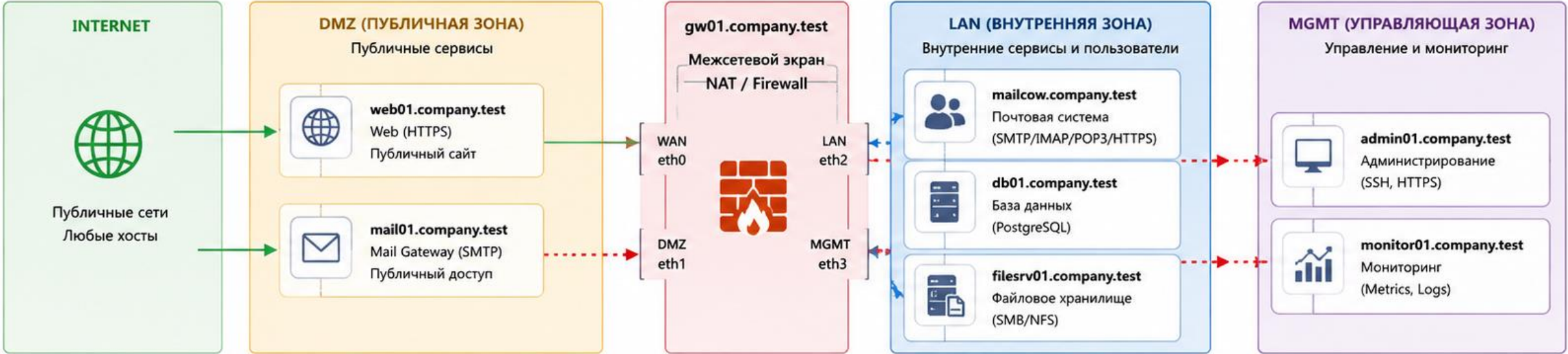
agents -> zabbix01 TCP 10051 ALLOW, if Active Checks are used

servers -> backup01 TCP 22 ALLOW, BorgBackup over SSH by selected backup model

Сетевые зоны и разрешенные firewall-потоки

company.test — сегментация и минимально необходимые соединения

- Разрешено
- Разрешено (исходящее)
- Запрещено (по умолчанию)



РАЗРЕШЕННЫЕ FIREWALL-ПОТОКИ

ИСТОЧНИК	НАЗНАЧЕНИЕ	ПРОТОКОЛ	ПОРТ	НАЗНАЧЕНИЕ (СЕРВИС)	НАПРАВЛЕНИЕ
INTERNET	web01.company.test	TCP	443	HTTPS (публичный сайт)	→
INTERNET	mail01.company.test	TCP	25	SMTP (входящая почта)	→
INTERNET	mail01.company.test	TCP	587	SMTP Submission	→
INTERNET	mail01.company.test	TCP	465	SMTPS	→
INTERNET	mail01.company.test	TCP	443	HTTPS (SOG, Autodiscover)	→
mail01.company.test (DMZ)	mailcow.company.test (LAN)	TCP	25	SMTP (прокси к Mailcow)	→
LAN	INTERNET	TCP	443	Исходящие обновления, API	← -
LAN	INTERNET	TCP	80	Исходящие обновления (HTTP)	← -
admin01.company.test (MGMT)	gw01.company.test	TCP	22	SSH (управление FW)	→
admin01.company.test (MGMT)	ВСЕ ВНУТРЕННИЕ (LAN)	TCP	22, 443, 3389	Администрирование (SSH/HTTPS/RDP)	→
monitor01.company.test (MGMT)	ВСЕ ВНУТРЕННИЕ (LAN)	TCP	9100, 9200, 161	Мониторинг (Metrics, Syslog, SNMP)	→

ПО УМОЛЧАНИЮ ЗАПРЕЩЕНО

- Прямые подключения из INTERNET в LAN (кроме опубликованных в DMZ)
- Любые входящие подключения в MGMT из INTERNET
- Между LAN и DMZ иницируемые с LAN (только ответы на установленные)
- Весь трафик, не описанный в таблице разрешений

ПРИМЕЧАНИЯ

- Весь входящий трафик из INTERNET принимается только на gw01 (NAT + Firewall).
- DMZ содержит только публичные сервисы и не имеет доступа к LAN, кроме необходимых.
- LAN иницирует исходящие соединения (обновления, репозитории, внешние API).
- MGMT изолирована и используется только для управления и мониторинга.
- Рекомендуется журналирование всех соединений и регулярный аудит правил.



Принципы: минимальные необходимые потоки, сегментация, управление из MGMT, нулевая доверенность по умолчанию

Шлюз/Firewall: gw01.company.test

WAN: eth0 | DMZ: eth1 | LAN: eth2 | MGMT: eth3

VPN acceptance проверяет Allow и Deny

Valid Certificate -> Tunnel UP

VPN Client -> web.company.test:443 -> ALLOW

VPN Client -> web01:22 -> DENY

Revoked Certificate -> Tunnel denied

Проверка VPN должна показывать не только подключение, но и ограничения доступа

Storage входит в обязательное ядро проекта

Создать separate LV

Создать filesystem

Настроить mount point

Прописать UUID в /etc/fstab

Проверить sudo mount -a

После reboot mount должен восстановиться
автоматически

Storage acceptance проверяется командами, а не обещанием

lsblk -f - увидеть устройство, filesystem и UUID

findmnt - проверить фактическую точку монтирования

df -hT - проверить размер и тип filesystem

sudo mount -a - проверить /etc/fstab без reboot

После reboot повторить findmnt и df -hT

RAID degraded/recovery - дополнительный

демонстрационный сценарий, а не обязательный тест

для всех

Git обязателен, git01 не обязателен

Git Repository - обязательная часть проекта

В Git хранятся configuration, scripts, docs и change records

git01 / Gitea / GitLab - дополнительный Remote Server

В базовой работе допустим локальный Git или remote, предоставленный преподавателем

Перед git add выполняется secret check

BorgBackup - базовый backup tool проекта

В проекте используется backup tool, изученный в модуле

Базовый вариант - BorgBackup

borg list - увидеть archives

borg info - проверить информацию о repository/archive

borg check - проверить целостность

Test Restore: archive -> /restore-test -> verification

RPO и RTO фиксируются во время восстановления

RPO - допустимая потеря данных

RTO - фактическое время восстановления сервиса или данных

Во время Test Restore студент фиксирует фактический RTO

Backup без Test Restore не считается полноценным результатом

Перед рискованным изменением рабочего сервиса должна существовать проверенная точка восстановления

Порядок развертывания снижает количество случайных ошибок

1. Architecture
2. Address Plan
3. Base OS
4. Routing / Firewall
5. DNS / Time / SSH
6. Git
7. Baseline Monitoring
8. Application Services
9. Backup
10. Test Restore, Failure Tests, Reboot Test, Documentation

Correct Time влияет на несколько служб

timedatectl status

Correct Time -> TLS

Correct Time -> VPN certificates

Correct Time -> Mail

Correct Time -> Logs

Correct Time -> Monitoring

Zabbix подключается рано для baseline

Baseline показывает нормальное состояние до сложных изменений

После изменения проверять Latest Data

Проверять Item freshness

Проверять Unsupported Items

Проверять Problems

Отсутствие Problem не является достаточной проверкой без external test

Диагностика начинается с клиентского пути

Client

DNS

Route

Firewall / VPN

Entry Point

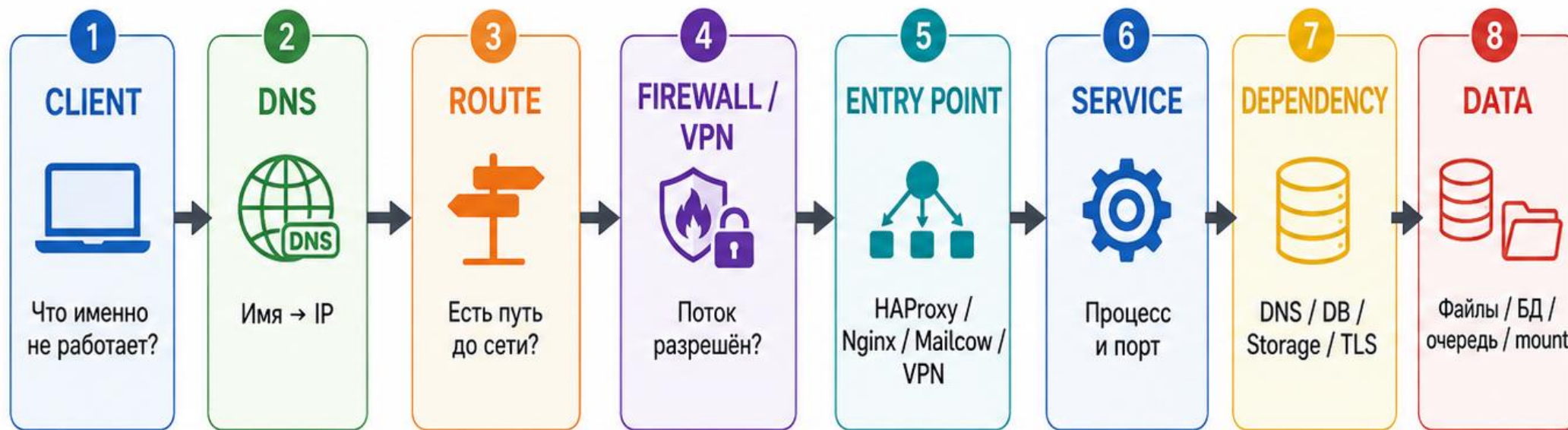
Service

Dependency

Data

Диагностика: от Client до Data

Последовательная проверка инфраструктурной проблемы



Основные инструменты

- `ping / curl`
- `dig / getent`
- `ip route / traceroute`
- `nft / ss / nc`
- `systemctl / journalctl`
- `df / lsblk / mount`



Не менять несколько параметров одновременно



Главный вывод:

Диагностика идёт по цепочке:
клиент → сеть → точка входа → сервис → зависимости → данные

External Test выполняется не только с сервера

`curl -I https://web.company.test`

Проверка выполняется с client01 или admin01

Проверка с самого web01 не доказывает
пользовательскую доступность

External Test должен проходить через DNS, route,
firewall, entry point и service

Именно внешний тест связывает архитектуру с
пользовательским результатом

HAProxy acceptance проверяет балансировку и отказ

web01 UP, web02 UP -> traffic distributed

web01 DOWN, web02 UP -> web remains available

web01 DOWN, web02 DOWN -> HTTP 503

Recovery должен закрывать Zabbix Problem

Проверки выполняются через web.company.test, а не напрямую к backend

Bash health check должен давать понятный статус

```
bash -n server-health.sh
```

```
./server-health.sh
```

```
echo $?
```

Сценарий должен возвращать OK, WARNING, CRITICAL или UNKNOWN, где применимо

Health check должен быть полезен для диагностики, а не просто печатать красивый текст

Backup acceptance требует restore verification

Archive exists

Repository check passes

Selected data can be restored

Restored service or data verified

Actual RTO recorded

Backup без восстановления в /restore-test не
принимается как полноценная защита данных

Zabbix acceptance проверяет полный цикл события

Normal

Failure

Problem

Acknowledge

Fix

Recovery

Latest Data должны оставаться свежими

Docker acceptance применяется только к Docker-варианту

Docker Application - расширенный компонент

Проверка: container recreated -> persistent data remains

Если Docker application не выбран, этот критерий не применяется

Не штрафовать проект без Docker за отсутствие Docker-specific checks

Если Docker выбран, persistence должна быть доказана, а не заявлена

Mailcow acceptance применяется только к Mailcow-варианту

Mailcow - расширенный компонент

Проверка: user1@company.test -> user2@company.test

Mail Flow должен завершаться успешно

Open Relay criterion применяется только к Mailcow project

Если Mailcow не выбран, mail-specific checks не входят в приемку

Reboot Test обязателен для всех проектов

После reboot проверить routes

Проверить firewall

Проверить mounts

Проверить DNS

Проверить OpenVPN, HAProxy и Zabbix Agent

Проверить backup-related mounts

Docker и Mailcow проверяются после reboot только если они используются

Проект, работающий только до reboot, не принимается

Временная route исчезает после reboot

Ручной firewall-rule исчезает после reboot

Неправильный mount без /etc/fstab не восстановится

Сервис без enable может не стартовать

Reboot Test выявляет временные настройки, которые маскируются во время демонстрации

Startup order после полного shutdown должен быть объясним

1. Router / Network
2. DNS
3. Data Dependencies
4. Application Servers
5. Entry Points
6. Clients

Monitoring и Backup запускаются независимо и не являются prerequisite пользовательского сервиса

Документация обязательна, но должна быть сильной

As-Built Network Diagram

Address + DNS Plan

Firewall Matrix

README / Operations Guide

Backup + Restore Procedure

Two Recovery Runbooks

One Change Record

Acceptance Checklist

Recovery Runbooks нужны минимум для двух сценариев

Web / HAProxy Failure - обязательный runbook

Backup Restore - обязательный runbook

Для остальных сервисов достаточно кратких recovery notes

В полном варианте можно добавить Mailcow, VPN или DNS runbooks

Runbook должен содержать symptoms, commands, expected result и rollback

Responsibility Matrix нужна только для командной работы

Для индивидуального проекта отдельная responsibility matrix не нужна

Для парной или командной работы она обязательна

Матрица показывает, кто отвечает за сеть, сервисы, backup, monitoring и documentation

Ответственность не отменяет общее понимание проекта

На защите каждый участник должен объяснить общую архитектуру

Секреты нельзя публиковать в сдаваемых артефактах

No Secrets In: Git

No Secrets In: Report

No Secrets In: Screenshots

No Secrets In: Logs

No Secrets In: PCAP

No Secrets In: Shell History

Использовать REDACTED, CHANGE_ME или
SECRET_STORED_SEPARATELY

Оценивание должно соответствовать требованиям

Architecture / Naming - 10

DNS - 10

VPN - 10

Web / HAProxy - 10

Storage + Bash/Git - 10

Backup / Restore - 15

Zabbix - 10

Extended Component - 10

Documentation - 10

Defense - 5

Extended Component выбирается осознанно

Mailcow

Docker App

Keepalived / VIP

Separate Backup Network

additional automation

Выбранный компонент должен иметь свою проверку,
документацию и критерий приемки

Critical Fail Conditions останавливают приемку проекта

No Test Restore

Real Secrets published

No Firewall Control

Cannot explain Address Plan

Cannot explain DNS Role

Cannot explain dependencies

Infrastructure fails after reboot

Условные критические ошибки зависят от выбранного варианта

Open Relay - только Mailcow project

Docker persistence failure - только Docker project

RAID degraded/recovery failure - только если RAID failure
выбран как расширенный сценарий

Keeralived/VIP failure - только если выбран HA-
балансировщик

Separate Backup Network mismatch - только если
выбран отдельный backup-сегмент

Главный сценарий защиты показывает работающую систему

1. Network / DNS
2. VPN
3. Firewall Allow/Deny
4. HAProxy balancing
5. Backend failure
6. Zabbix Problem/Recovery
7. Borg Test Restore
8. Reboot Test

Расширенный компонент показывается
дополнительно



Матрица приёмки итогового Linux-проекта

Что нужно показать на защите и какой результат считается успешным



Компонент		Что проверить	Критерий ОК
	1 DNS / Имена	Host Name, Service Name, A-записи	✓ web.company.test и host-имена разрешаются корректно
	2 VPN	Valid client, revoked client	✓ Разрешённый клиент подключается, отозванный — нет
	3 Firewall	Allow / Deny по матрице	✓ Нужные потоки доступны, запрещённые — блокируются
	4 Web / HAProxy	Балансировка и отказ backend	✓ 2 backend UP — OK; 1 backend DOWN — сервис доступен; 0 backend — 503
	5 Storage	UUID, mount, fstab	✓ Точки монтирования восстанавливаются после reboot
	6 Bash / Git	Health-check script и версия конфигураций	✓ Скрипт работает, изменения зафиксированы в Git
	7 Backup / Restore	Архив, test restore, RTO	✓ Есть успешное тестовое восстановление
	8 Zabbix	Problem → Acknowledge → Recovery	✓ Событие фиксируется и корректно закрывается
	9 Reboot Test	Полная перезагрузка стенда	✓ После reboot инфраструктура снова работает



**Проект
принят,
если**

- Работают сервисы и зависимости
- Проверены отказ и восстановление
- Есть test restore
- Есть документация



**Критические
ошибки**

- Нет test restore
- Секреты в Git / отчёте
- Нет контроля firewall
- Проект ломается после reboot



Главный вывод: итоговый проект оценивают не по количеству установленных сервисов, а по управляемости, диагностике, восстановлению и повторяемости результата.

Итоги лекции

Итоговый проект проверяет связанную инфраструктуру, а не количество установленных пакетов

gw01, DNS, address plan и firewall matrix делают сеть управляемой и проверяемой

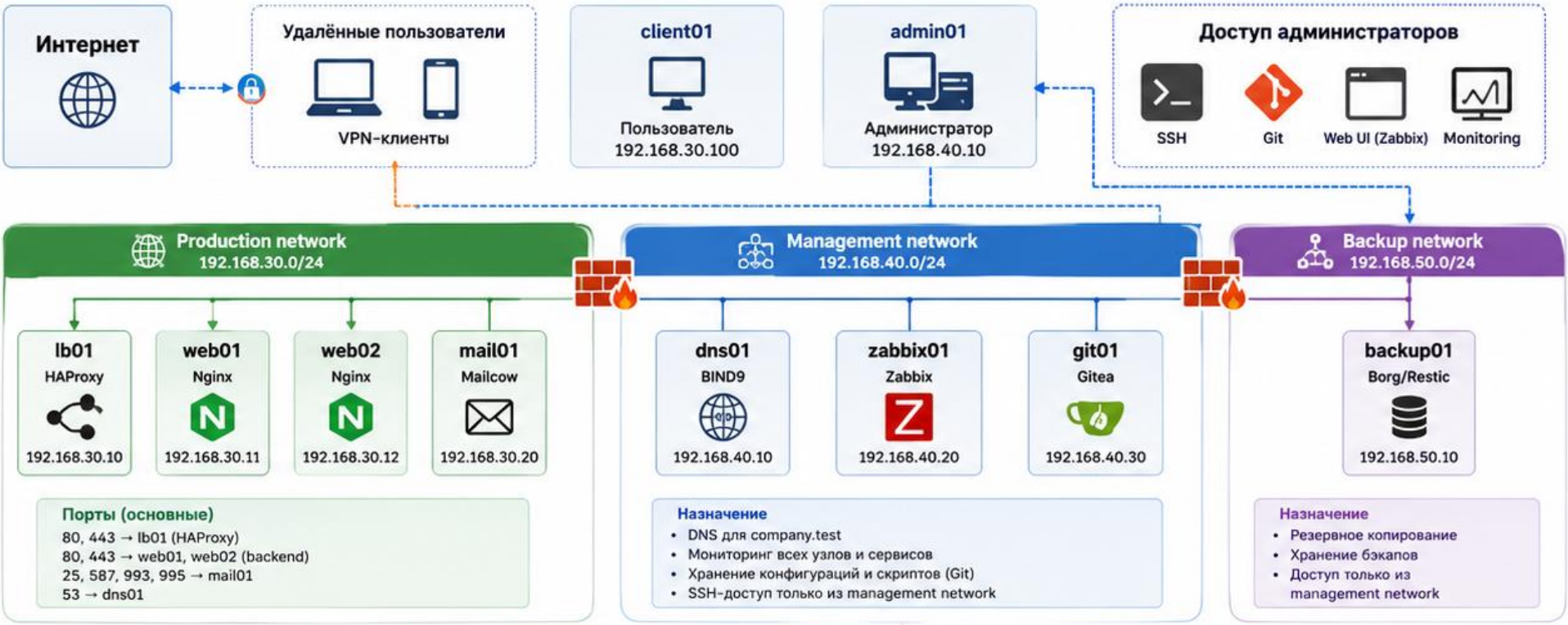
Storage, Git, BorgBackup, Zabbix и documentation входят в обязательное ядро

Docker, Mailcow, Keepralived и Separate Backup Network являются расширенными вариантами

Проект принимается только после external tests, failure tests, test restore и reboot test

Топология: итоговая архитектура Linux-инфраструктуры

company.test – связанная, безопасная и отказоустойчивая Linux-инфраструктура



Naming standard

<роль><номер>.company.test

Примеры: web01.company.test, dns01.company.test

Адресный план

Production network	192.168.30.0/24
Management network	192.168.40.0/24
Backup network	192.168.50.0/24
VPN subnet	10.8.0.0/24

- Ключевые сервисы**
- DNS (dns01)**
Внутренний DNS company.test
 - VPN (vpn01)**
OpenVPN для удалённого доступа
 - Load Balancer (lb01)**
HAProxy, балансировка и TLS
 - Web backend (web01, web02)**
Веб-приложение (Nginx/PHP/Node.js)
 - Mail (mail01)**
Почта и антиспам (Mailcow)
 - Monitoring (zabbix01)**
Zabbix: мониторинг и алерты
 - Backup (backup01)**
Резервное копирование (Borg/Restic)
 - Git (git01)**
Git сервер для конфигураций и скриптов

- Отказоустойчивость и восстановление**
- LB01 может быть дублирован (Keepalived) для VIP и отказоустойчивости
 - WEB01/WEB02 – горизонтальное масштабирование
 - Еженедельное тестирование восстановления из бэкапа
 - Документация со схемами, адресами, правилами и процедурами

Единые принципы: именование, адресация, разделение сетей, минимальные права, мониторинг, резервное копирование, документация



Схема: матрица зависимостей итогового стенда

company.test — взаимосвязи сервисов, данных и сетей

Как читать матрицу

- Зелёная галочка**
Сервис в строке использует сервис в столбце
- Оранжевый круг**
Сервис в строке зависит от данных/ресурсов в столбце
- Фиолетовая стрелка**
Необязательная или условная зависимость
- Серая ячейка**
Нет прямой зависимости

Ключевые принципы

- Минимизация точек отказа
- Чёткий порядок запуска
- Мониторинг и алерты
- Резервное копирование
- Документирование и тесты

Зависит от →

Использует ↓

	DNS dns01	VPN vpn01	HAProxy lb01	Web web01/02	Mail mail01	Zabbix zabbix01	Backup backup01	Git git01	Storage (данные)	ОС и сеть (базовые)
DNS dns01	—	—	—	—	—	—	—	—	—	
VPN vpn01		—	—	—	—	—	—	—	—	
HAProxy lb01			—	—	—	—	—	—	—	
Web web01/02				—	—	—	—	—		
Mail mail01			—	—	—	—	—	—		
Zabbix zabbix01			—	—	—	—	—	—		
Backup backup01			—	—		—	—	—		
Git git01			—	—	—	—	—			
Storage (данные)	—	—	—	—	—	—	—		—	
ОС и сеть (базовые)										—

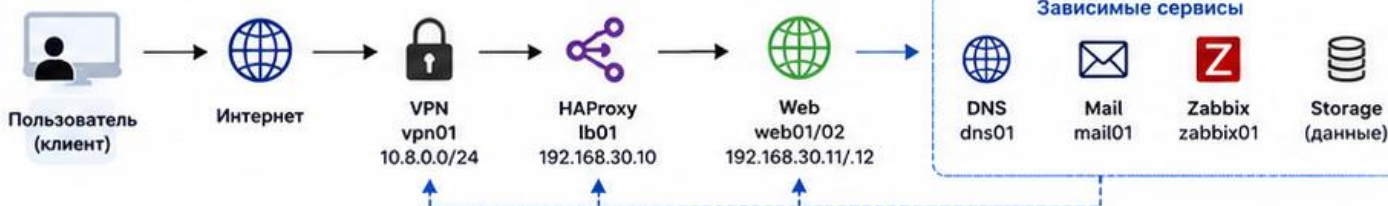
Примеры зависимостей

- Web → HAProxy**
Веб-серверы получают трафик только через балансировщик lb01
- Mail → DNS**
Почтовый сервер использует DNS для MX-записей и поиска доменов
- Backup → Storage**
Система резервного копирования читает и записывает данные в хранилище
- Администрирование → VPN**
Доступ администраторов к серверам только через VPN-шлюз

Порядок запуска (рекомендуемый)

- ОС и сеть (базовые настройки)
- DNS
- VPN
- HAProxy
- Web (web01, web02)
- Mail (если используется)
- Zabbix
- Storage и данные
- Backup
- Git и вспомогательные сервисы

Поток доступа пользователя



Условные обозначения (в матрице)

- Прямая зависимость (нужен для работы)
- Использует данные/ресурсы
- Необязательная / условная зависимость
- Нет прямой зависимости

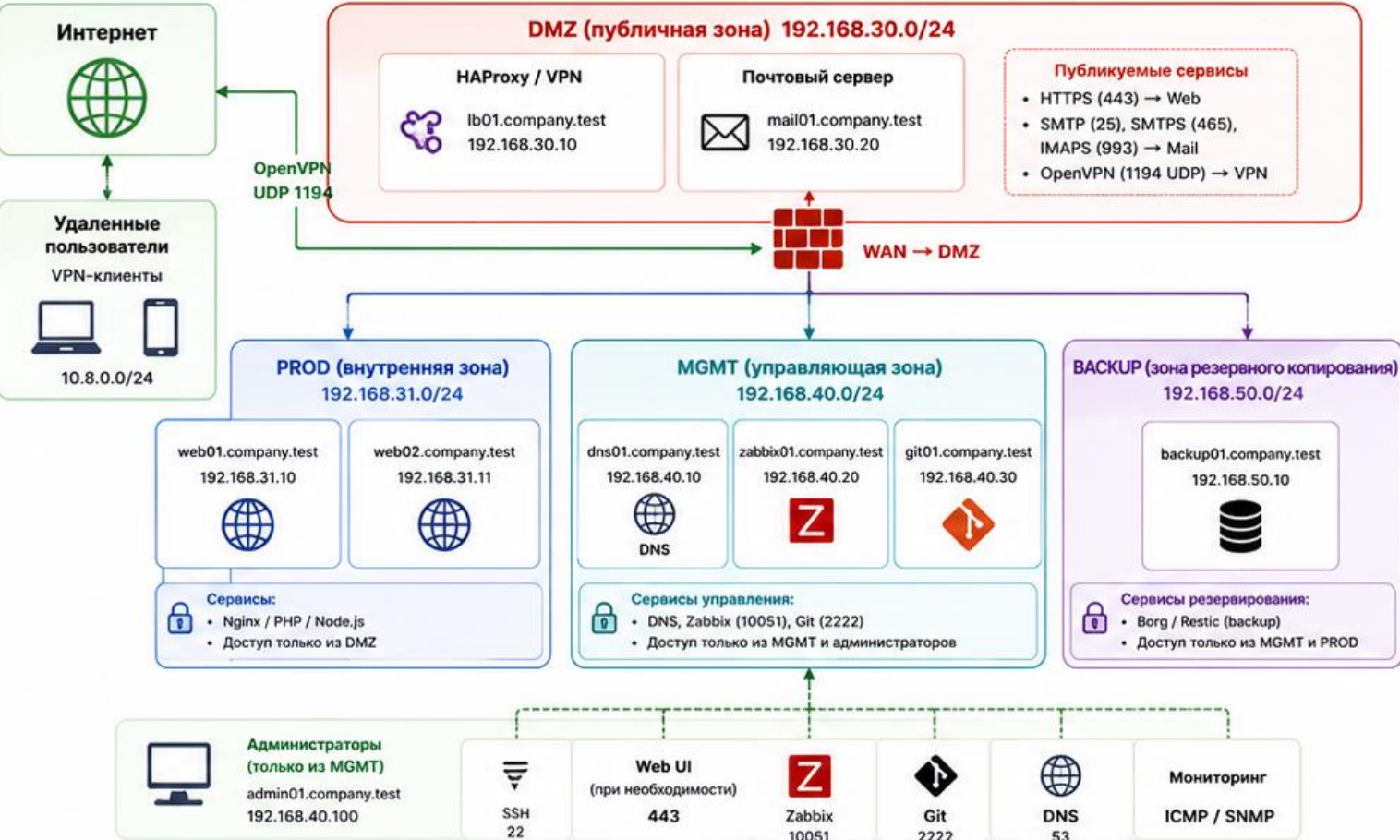
Важно

- Изменения в одной службе могут повлиять на другие.
- Проверяйте зависимости перед обновлениями.
- Всегда делайте backup перед изменениями.

Матрица помогает понять, какие сервисы критичны, в каком порядке их разворачивать и что проверять при ошибках или отказах.

Схема: сетевые зоны и разрешенные firewall-потоки

company.test — разделяем сети, разрешаем только необходимое



Разрешенные firewall-потоки (минимально необходимые)					
№	Источник (зона)	Назначение (зона/узел)	Сервис / Порт	Протокол	Назначение
1	Интернет	DMZ (lb01)	UDP 1194	UDP	OpenVPN доступ
2	Интернет	DMZ (lb01)	TCP 443	TCP	HTTPS к web
3	Интернет	DMZ (mail01)	TCP 25	TCP	SMTP
4	Интернет	DMZ (mail01)	TCP 465	TCP	SMTPS
5	Интернет	DMZ (mail01)	TCP 993	TCP	IMAPS
6	DMZ (lb01)	PROD (web01, web02)	TCP 80, 443	TCP	HTTP/HTTPS к веб
7	DMZ (mail01)	PROD (web01, web02)	TCP 80, 443	TCP	Web интерфейсы (если нужны)
8	MGMT (admin01)	MGMT (все узлы)	TCP 22	TCP	SSH управление
9	MGMT (admin01)	MGMT (zabbix01)	TCP 10051	TCP	Zabbix агент
10	MGMT (admin01)	MGMT (git01)	TCP 2222	TCP	Git доступ
11	MGMT (dns01)	Все зоны	TCP/UDP 53	TCP/UDP	DNS-запросы
12	PROD (web01,02)	BACKUP (backup01)	TCP 22, 2333*	TCP	Резервное копирование (Borg/Restic по SSH)
13	MGMT (admin01)	BACKUP (backup01)	TCP 22	TCP	Управление backup
14	Zabbix (zabbix01)	Все узлы	ICMP / SNMP	ICMP/UDP	Мониторинг
*	Любой	Любой	Остальные	—	ЗАПРЕЩЕНО (DROP)

Политика по умолчанию: запрещать все, разрешать только явно указанные потоки.
Firewall установлен на lb01 (DMZ) и/или на границе сетей (межсетевой экран).

Сетевые зоны

- DMZ (публичная) – сервисы, доступные из Интернета
- PROD (внутренняя) – рабочие сервисы компании
- MGMT (управляющая) – администрирование и мониторинг
- BACKUP (резервная) – хранение резервных копий
- VPN subnet – адреса подключенных клиентов

Принципы разделения

- Публичные сервисы только в DMZ
- Веб-бэкэнды недоступны из Интернета напрямую
- Управление только из MGMT сети
- Резервные копии изолированы в BACKUP сети
- Минимально необходимые порты и протоколы

Ключевые правила

- Не открываем лишние порты
- Используем TLS для всех чувствительных сервисов
- SSH доступ только из MGMT и по ключам
- Регулярно проверяем логи firewall и сервисов

Быстрая проверка потоков

```
# Проверка из Интернета
nmap -Pn lb01.company.test

# Проверка из PROD к backup
nc -zv backup01.company.test 22

# Проверка DNS
nslookup web01.company.test dns01.company.test
```

Обозначения

- Разрешено
- Публичный доступ
- Внутренний доступ
- Только управление
- Backup-потоки
- Межсетевой экран



Диаграмма: безопасный порядок развертывания итогового стенда

company.test — сначала проектирование и наблюдаемость, потом сервисы и проверка

1. Проектирование



- Роли серверов
- Имена и FQDN
- Адресный план
- Сетевые зоны
- Сценарии приемки

2. Базовая ОС и доступ



- Установка ОС
- Hostname и IP
- SSH-доступ
- Время / NTP
- DNS и маршруты

3. Подготовка к изменениям



- Git для конфигураций
- Backup перед изменением
- Monitoring и Zabbix
- Health checks
- Secret check

4. Базовые инфраструктурные сервисы



dns01



vpn01



lb01 / HAProxy



Storage / mount



Firewall rules

5. Сервисы приложений



web01 / web02



Mailcow / Docker



Git service



Backup service



Только после наблюдаемости

6. Проверка после изменений



Syntax check



Проверка портов



Свежесть метрик



External test



Rollback при ошибке

7. Приемка и сопровождение



- Acceptance tests
- Failover test
- Reboot test
- Test restore
- Документация



А. Ключевой принцип

- ✓ Не ставим сложные сервисы первыми
- ✓ Наблюдаемость раньше изменений
- ✓ Backup без restore — не гарантия
- ✓ Документация должна совпадать со стендом



В. Перед Git commit

- ☐ git diff
- ☐ syntax check
- ☐ секреты не попали в файлы
- ☐ есть backup
- ☐ понятный commit message



С. Финальная проверка

- ✓ DNS работает
- ✓ VPN дает только разрешенный доступ
- ✓ Web доступен через lb01
- ✓ Zabbix показывает актуальные данные
- ✓ Стенд переживает перезагрузку

Безопасный цикл изменения



Изменение



Проверка



Мониторинг



Backup/Restore



Документация



Диаграмма: диагностика от клиента до данных

company.test — пошаговый путь поиска проблемы: от жалобы пользователя до сервиса, зависимостей и данных



Главный принцип

- 1 Идём от client path, а не от случайной перезагрузки
- 2 Сначала подтверждаем симптом
- 3 Потом проверяем ресурсы, сеть, службу и данные
- 4 После исправления обязательно повторяем пользовательский тест

1. Симптом у клиента



- Подтвердить жалобу
- Что именно недоступно?
- Когда началось?

2. DNS



- Имя резолвится?
- Проверка: `resolvectl query`
- `dig / nslookup`

3. Сеть / маршрут / firewall / VPN



- IP и маршрут
- Есть ли VPN?
- Открыт ли нужный порт?
- `nftables / ss`

4. Точка входа



- Доступен ли lb01?
- Отвечает ли HTTPS?
- Есть ли балансировка?

5. Служба / приложение



- `systemctl status`
- `journalctl`
- `nginx -t / haproxy -c`
- `docker compose ps`

6. Зависимости



- Какие сервисы нужны?
- Нет ли скрытой причины?
- Проверить цепочку зависимостей

7. Данные / хранилище



- Смонтирован ли том?
- Есть ли место?
- `df -h / df -i`
- Backup и restore

Базовые проверки ОС



`uptime` — load average



`free -h` — RAM / swap



`df -h` — место на ФС



`df -i` — inode



`systemctl --failed` — отказавшие службы

Команды по этапам



DNS

`resolvectl query lb01.company.test`



Network

`ip -br addr`

`ip route`

`ss -ltnup`

`nft list ruleset`



Service

`systemctl status nginx`

`journalctl -u nginx -n 50`

`nginx -t`

`haproxy -c -f /etc/haproxy/haproxy.cfg`



External test

`curl -I https://web.company.test`



Ожидаемый результат

- ✓ DNS-имя резолвится
- ✓ Маршрут и порт доступны
- ✓ `curl` даёт 200 / 301 или ожидаемый статус
- ✓ Сервис работает, данные доступны



Алгоритм исправления



Симптом



Проверка



Локализация



Исправление



Повторный тест



В отчёте фиксируются: симптом, гипотеза, команда, результат, исправление, подтверждение



Инфографика: матрица приемки итогового Linux-проекта

company.test — что именно проверяется на защите, по каким критериям и что ведёт к непринятию проекта



1. Основные блоки оценки

Блок	Что проверяют	Баллы
Архитектура и naming	Роли, адресный план, DNS, зависимости	10
Базовые сервисы	DNS, VPN, Web/HAProxy, Bash/Git, Storage	по 10
Backup и Restore	Backup, test restore, RPO/RTO	15
Monitoring / Zabbix	Hosts, items, triggers, свежесть данных	10
Документация и защита	Схемы, matrix, runbooks, объяснение решений	5

Расширенная часть (Docker / Mailcow и др.) может оцениваться отдельно.

2. Приемочные тесты штатной работы

- DNS lookup всех узлов возвращает адреса из address plan
- VPN с valid certificate поднимает tunnel
- Web открывается через lb01
- HAProxy распределяет трафик между web01 и web02
- server-health.sh даёт понятный output
- Mailcow доставляет письмо user1 → user2 (если входит в вариант проекта)

3. Приемочные тесты безопасности

- VPN user видит только разрешённые ресурсы
- SSH на backend запрещён из user/VPN network
- Revoked certificate не подключается
- Management UI доступны только из admin01 / MGMT
- Firewall matrix совпадает с реальными правилами
- В Git и отчёте нет реальных секретов

4. Тесты отказа и восстановления



5. Критические условия непринятия

- Нет проверенного restore или студент не может его объяснить
- Секреты опубликованы в Git, отчёте или скриншотах
- Open Relay / лишний admin-доступ / отсутствующий firewall control
- Невозможно объяснить address plan, DNS role или зависимости
- Сервисы работают случайно и не переживают reboot

Что сдаёт студент

- Схема сети
- Address plan
- Firewall matrix
- Runbooks / Change Record

Финальный вывод



Проект принимается только тогда, когда он работает как связанная система, проходит тесты и может быть восстановлен по документации.

Контрольные вопросы

1. Почему нужно явно определить DNS Server?
2. Кто маршрутизирует traffic между сетевыми зонами?
3. Чем Production Traffic отличается от Management и Backup?
4. Почему отсутствие Zabbix Problems не доказывает исправность?
5. Что проверяется перед Git Commit?
6. Почему Backup без Restore недостаточен?
7. Как проверить VPN Allow и Deny?
8. Для чего нужен Reboot Test?
9. Что должно содержаться в Firewall Matrix?
10. Какие ошибки приводят к неприятию проекта?